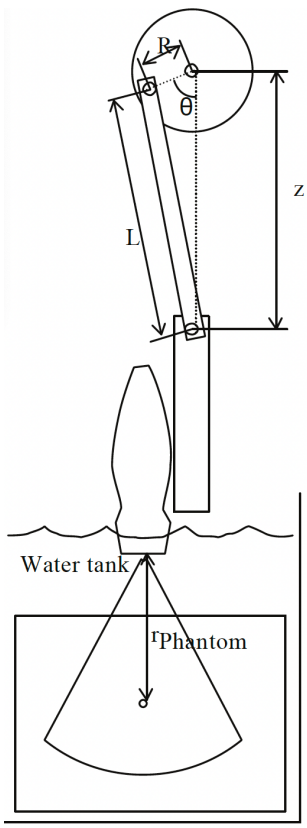
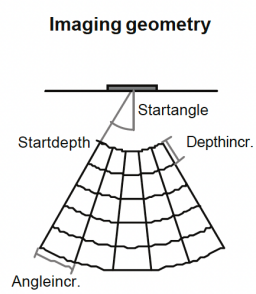
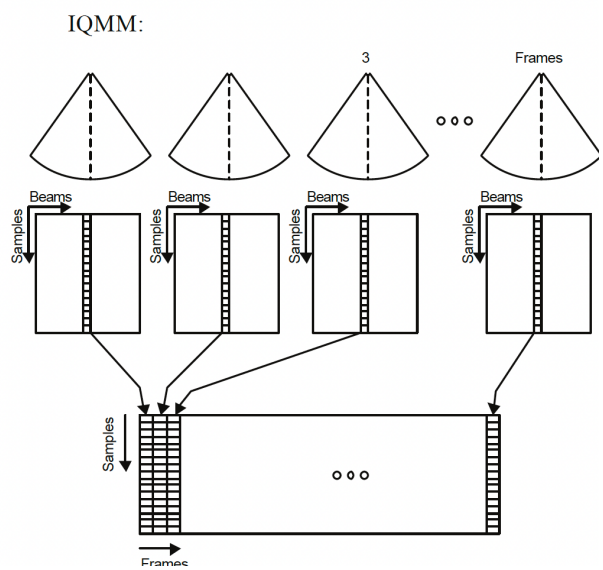
Exercise 7 – Motion and spectrum analysis

In this exercise we will start to work with ultrasound imaging of moving targets, and the estimation of velocities as used in blood flow imaging. Central to measuring velocity and displacement with ultrasound is the use of power spectral estimation. We will here use a lab dataset with motion to investigate different aspects of spectrum estimation.

The dataset was acquired in the ultrasound lab (many years ago) and is illustrated in the picture below. The ultrasound transducer is positioned to a rod and moved up and down in a cyclic manner (as seen left in the figure). A special recording setup images a stationary target in the water tank. The image geometry is depicted in the top right part of the figure. In this exercise, we will work with one-dimensional M-mode imaging and spectral-Doppler estimation, and to do this we will extract the center beam and plot this over time, as shown in the bottom right figure.







1. Load and plot the M-mode data

Download the datasets called *slowmotion.mat*. You can read this dataset in Matlab directly or through the *scipy.io.loadmat* function in Python. The data contains both IQ-data and various parameters describing the geometry and setup. From Python these are available in the dictionary that is made by *loadmat*, where 'iq' is the IQ data, while 'pars' contains relevant parameters.

- The dimensions in the IQ data are: [range/depth sample, beam number, frame number].
- Relevant parameters included are:

  'fps': frame rate, the time axis can be made based on this

  'depth_start': The starting depth of the scan in [m]

  'depth_inc': The increment between depth samples in [m]

  'f0': The center frequency of the ultrasound pulse

- To read the parameters in Python, scipy.io.loadmat uses structured Numpy arrays, the parameters can be read as:

  data = scipy.io.loadmat(filename)

  pars = data['pars']

  fps = pars[0,0]['fps'].item()

  etc.

  (The item() is to flatten the returned arrays to scalars, needed to avoid Numpy warnings when using it.)

a) Extract the center beam of the IQ data and process this to an M-mode. Display it as an image in the log-domain using a greyscale colormap, with correct depth and time axis. Limit the dynamic range to 40dB and use depth and time axis calculated from the parameters.

   Tips:

   - Normalize and set vmin=-40, and vmax=0 in the imshow function in Python, or use the clim function in Matlab.
   - Create the depth and time axis using the parameters above (e.g. using the Numpy arange function in Python, or ':' operator in Matlab.

b) What kind of motion do you observe?

In the lab setup, assuming R << L, it can be shown that the distance between a fixed point and the probe varies approximately as:

$$r(t) \approx -R \cdot cos\left(\frac{2\pi(t-t_0)}{T}\right) + z_0.$$

Where R is the excursion radius [m], t0 is the time offset [s], T is the rotation period [s], and z0 is the depth offset relative to the probe surface [m].

c) Estimate the rotation period *T*, the time offset *t0*, and the excursion radius *R* from the M-mode image. Insert the values into the analytic expression, and plot r(t) on top of the M-mode. Position at z_0 to overlap with the brightest structure in the M-mode. Can you make the calculated movement fit the observed movement? Note the values.

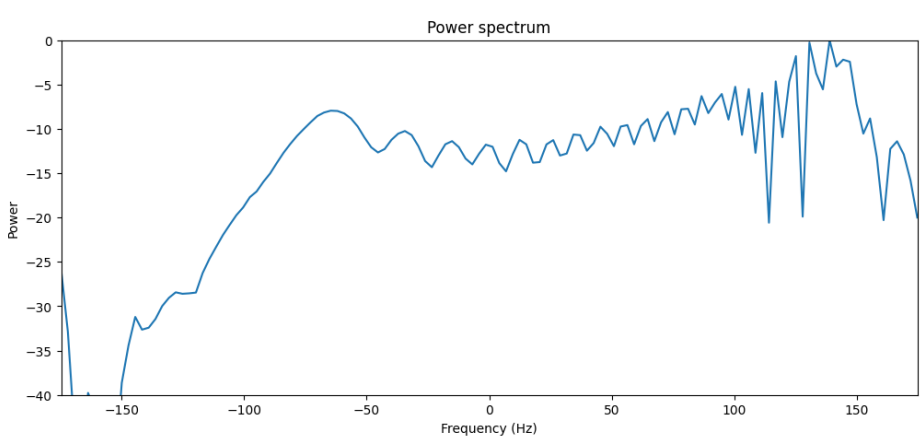
2. Spectral analysis

We will now do spectral analysis of the motion pattern in 1). We have observed that the motion pattern is cyclic and thus the velocity changes over time, i.e. it is a non-stationary / accelerated motion pattern.

a) **Spectral analysis:** Extract an *IQ time series* corresponding to one depth z_0 of the scatterer as found in 1c. Find the power spectrum estimate as (pseudocode):

```
P(w) = fftshift(abs(fft(IQ))^2)          # Spectral estimate
P_log(w) = 10*log10(P(w))                # Log-domain
P_log_n(w) = P_log(w) – max(P_log(w))    # Normalize
```

Plot the spectral estimate (log-domain, normalized) with correct frequency axis and use a dynamic range of 40dB (-40 to 0 dB). It should look something like the figure below.



What happened: You have now done spectral analysis along the complete time axis and thus smear all the variation in velocity over time into one plot. This is not so useful in our case where the velocity is changing over time, but it brings us to an **important note:** *Power spectral analysis is (as a starting point) assuming a stationary signal*.

Important trick: When you have few samples in your signal and take the FFT with the same length as the signal, the resulting spectrum can look very discrete, i.e. with 16 time samples as input we get 16 frequencies as output. To make more pleasing plots, set the number of frequency components (*Nfft*) to calculate to something higher than the length of the signal (see fft documentation for Numpy or Matlab). Typically, we use Nfft of 2^N, e.g. 32, 64, 128, 256, etc. The effect is an interpolation of the spectrum, which looks much more pleasing to the eye and helps us analyze. *Be sure to use this trick in the following analysis*.

b) **Time-varying spectral analysis:** The simplest way to do spectral analysis for a non-stationary signal is to divide the time series into portions / windows to get a more localized spectrum estimate and thus the distribution of velocities in the data around a given time point. We often refer to this portion as the *observation window*. By reducing the observation window, we can get more localized spectral estimates in time, however, at the same time we reduce the number of data/samples as input to our estimator. This leads to a trade-off, as the number of samples and frequency resolution are inversely proportional. The effect will become apparent in the following.

   Extract a portion of *Nt=16* samples anywhere in the time series, estimate the power spectrum, and plot as above. *Did you get a more localized velocity distribution?* What else can you observe?

   You can also try with other values for the number of samples and see, but before we move on, let's improve on our spectral estimator to make it easier to in the end choose a suitable observation window, and achieve better results.

c) **Windowing:** We can improve our basic spectral estimator by using a smooth window function. Our time series is of finite length, which can be seen as if a rectangular window is multiplied with our time series, capping it at its ends. When we estimate the spectral content, our estimate is effectively a combination of this window and the spectral content in the signal. A rectangular window provides the best frequency resolution since we use all our signal, but the worst window side-lobe suppression. A better choice is often the Hamming window, which provides a decent trade-off between the two. Let's have a look.

Python: Import *boxcar* and *hamming* window functions from scipy.signal.windows
Matlab: Use the *boxcar* or *hamming* functions

Multiply your signal (the extracted observation window) with the smooth window function before you do the spectral analysis. Plot the resulting spectrum for a window length of 16 with boxcar and Hamming window. What do you observe?

d) **Time-varying spectral analysis 2:** Using the Hamming window, let's try to find a suitable observation window for our data. Remember, this is in the end a trade-off between resolution in time and frequency/velocity.

Do windowed spectral analysis with 8, 16, 32, 64, 128 samples. What do you observe? Can you propose a suitable window for our analysis?

e) **Sonogram display:** Let's make a time-varying display of our spectral analysis, which we often call a *sonogram*. Step through the signal one sample window at a time (as you selected above). For each step estimate the power spectrum and store this in a matrix. In the end, plot this matrix as an image. Use a grey scale colormap and restrict the dynamic range to 40 dB as before. You should be able to see a time-varying spectrum following the expected motion as the change in position (velocity) seen in the M-mode. Play with different values of the window length to see the effect of the trade-off between time and frequency resolution. Did you change your choice in d)?

f) **Comparison to analytical solution:** What is the analytical velocity? By taking the temporal derivative of the analytical displacement function r(t):

$$v(t) = r'(t) = \frac{2\pi R}{T} \cdot sin\left(\frac{2\pi(t - t_0)}{T}\right)$$

One more thing is needed: The Doppler equation can be used to convert from expected (radial) velocity to frequency: $f_d = -\frac{2f_0 v}{c}$, where f0 is given in the parameters (f0=2.5MHz), and c=1490 (speed of sound in water).

Enter the values for the parameters to get the velocity over time and convert to frequency. Finally, plot the analytical solution of the velocity vs. time over the sonogram. *Does it match the sonogram?*